Report: Identifying Key Entities Recipe Data

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Import Libraries

1.1. Installation of sklearn-crfsuite

The sklearn-crfsuite library uses the Conditional Random Fields to train them for problems like POS tagging for feature-based representations and supports efficient inference.

```
# installation of sklearn_crfsuite
!pip install sklearn_crfsuite==0.5.0
```

```
Collecting sklearn_crfsuite==0.5.0
  Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl.metadata (4.9 kB)
Collecting python-crfsuite>=0.9.7 (from sklearn_crfsuite==0.5.0)
  Downloading python_crfsuite-0.9.12-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (4.3 kB)
Requirement already satisfied: scikit-learn>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from sklearn_crfsuite==0.5.0) (1.6.1)
Requirement already satisfied: tabulate>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from sklearn_crfsuite==0.5.0) (0.9.0)
Requirement already satisfied: tqdm>=2.0 in /usr/local/lib/python3.12/dist-packages (from sklearn_crfsuite==0.5.0) (4.67.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn_crfsuite==0.5.0) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn_crfsuite==0.5.0) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn_crfsuite==0.5.0) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn_crfsuite==0.5.0) (3.6.0
Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl (10 kB)
Downloading python_crfsuite-0.9.12-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 9.3 MB/s eta 0:00:00
Installing collected packages: python-crfsuite, sklearn_crfsuite
Successfully installed python-crfsuite-0.9.12 sklearn_crfsuite-0.5.0
```

1.2. Import necessary libraries

The libraries in the following screenshots are necessary for the code. The *pd.set_option()* function is used to control how the DataFrames are displayed to make the output easy to read.

```
# Import warnings
import warnings
warnings.filterwarnings('ignore')
```

```python
# Import necessary libraries
import json  # For handling JSON data
import pandas as pd  # For data manipulation and analysis
import re  # For regular expressions (useful for text preprocessing)
import matplotlib.pyplot as plt  # For visualisation
import seaborn as sns  # For advanced data visualisation
import sklearn_crfsuite  # CRF (Conditional Random Fields) implementation for sequence modeling
import numpy as np  # For numerical computations
# Saving and loading machine learning models
import joblib
import random
import spacy
from IPython.display import display, Markdown # For displaying well-formatted output

from fractions import Fraction  # For handling fractional values in numerical data
# Importing tools for feature engineering and model training
from collections import Counter  # For counting occurrences of elements in a list
from sklearn.model_selection import train_test_split  # For splitting dataset into train and test sets
from sklearn_crfsuite import metrics  # For evaluating CRF models
from sklearn_crfsuite.metrics import flat_classification_report
from sklearn.utils.class_weight import compute_class_weight
from collections import Counter
from sklearn.metrics import confusion_matrix
```

```python
# Ensure pandas displays full content
pd.set_option('display.max_colwidth', None)
pd.set_option('display.expand_frame_repr', False)
```

2. Data Ingestion and Preparation

2.1. Read Recipe Data from DataFrame and Prepare the Data for Analysis

2.1.1. Define a *load_json_dataframe* function

1) The function *load_json_dataframe()* takes an input parameter which is the path of the file.
2) The *open()* function opens the file at the specified location in read mode. The *with* keyword ensures that the file will be closed properly after reading with a file objects used to read the data

```python
# define a function to load json file to a dataframe
from google.colab import drive
drive.mount('/content/drive')

def load_json_dataframe(file_path):
    with open(file_path, 'r') as f:
        data = json.load(f)
    return pd.DataFrame(data)
```

```
Mounted at /content/drive
```

2.1.2. Execute the *load_json_dataframe* function

The initial JSON file is read and a DataFrame is created, using the function that was defined in the previous section.

```
# read the json file by giving the file path and create a dataframe
df = load_json_dataframe('/content/drive/MyDrive/Assignment 5/ingredient_and_quantity.json')
```

2.1.3. Describe the DataFrame

1) Using the *.head()* function

The first 5 rows of the DataFrame are displayed using the *df.head()* function. There is no parameter specified in it as the default number of rows that are displayed are 5.

```
# display first five rows of the dataframe - df
df.head()
```

input	pos
6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient
1-1/2 cups Rice Vermicelli Noodles Thin 1 Onion sliced 1/2 cup Carrots Gajjar chopped 1/3 Green peas Matar 2 Chillies 1/4 teaspoon Asafoetida hing Mustard seeds White Urad Dal Split Ghee sprig Curry leaves Salt Lemon juice	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient quantity unit ingredient ingredient ingredient quantity ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient
500 grams Chicken 2 Onion chopped 1 Tomato 4 Green Chillies slit inch Ginger finely 6 cloves Garlic 1/2 teaspoon Turmeric powder Haldi Garam masala tablespoon Sesame Gingelly Oil 1/4 Methi Seeds Fenugreek Coriander Dhania Dry Red Fennel seeds Saunf cups Sorrel Leaves Gongura picked and	quantity unit ingredient quantity ingredient ingredient quantity ingredient quantity ingredient ingredient ingredient unit ingredient ingredient quantity unit ingredient quantity unit ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
1 tablespoon chana dal white urad 2 red chillies coriander seeds 3 inches ginger onion tomato Teaspoon mustard asafoetida sprig curry	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient unit ingredient ingredient unit ingredient

From the output above, we see that the *input column* contains the raw text data where each row is a long sentence. The *pos column* in the same output contains token-level labels that are corresponding to the words, already present, in the input column.

2) Using the *.shape* attribute

The attribute is used to check the size of the DataFrame and the output is always in a tuple format.

```
# print the dimensions of dataframe - df
df.shape
```

```
(285, 2)
```

Based on the output, we see that there are 285 rows in the DataFrame and each row represents one observation. There are 2 columns present and each column represents a variable or a field which in this case is “input” and “pos”.

3) Using the *.info()* function

The function gives the summary of the DataFrame to see how many rows and columns exist, if there are any values are missing and knowing the data type of each column.

```
# print the information of the dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 285 entries, 0 to 284
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   input   285 non-null    object
 1   pos     285 non-null    object
dtypes: object(2)
memory usage: 4.6+ KB
```

Based on the output, we see that the object type is a DataFrame with 285 rows and 2 columns. We also see that there are no nulls in the dataset and that the "object" datatype refers to the columns storing string/text values.

2.2. Recipe Data Manipulation

2.2.1. Create input_tokens and pos_tokens columns by splitting the input and pos from the dataframe

1) The 'input' column in the DataFrame is selected and *.apply()* is used to process each row at a time.
2) The *lambda* keyword is an anonymous function where 'x' represents one sentence from the column.
3) The *.split()* function is then used to split the sentence on the spaces present and then converts into a list of words, also known as tokens. These tokens are stored in a new column called 'input_tokens'.
4) Similarly, the steps are done for the POS column, creating its own list of tokens. These tokens are stored in a new column called 'pos_tokens'.
5) Finally, the first 5 rows of the dataset, with the two new columns are displayed using the *.head()* function.

```
# split the input and pos into input_tokens and pos_tokens in the dataframe

# Tokenize input
# Tokenize POS
df['input_tokens'] = df['input'].apply(lambda x: x.split())
df['pos_tokens'] = df['pos'].apply(lambda x: x.split())
```

```python
# display first five rows of the dataframe - df
df.head()
```

input	pos	input_tokens	pos_tokens
6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[6, Karela, Bitter, Gourd, Pavakkai, Salt, 1, Onion, 3, tablespoon, Gram, flour, besan, 2, teaspoons, Turmeric, powder, Haldi, Red, Chilli, Cumin, seeds, Jeera, Coriander, Powder, Dhania, Amchur, Dry, Mango, Sunflower, Oil]	[quantity, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, quantity, unit, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]
2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient quantity unit ingredient ingredient	[2-1/2, cups, rice, cooked, 3, tomatoes, teaspoons, BC, Belle, Bhat, powder, 1, teaspoon, chickpea, lentils, 1/2, cumin, seeds, white, urad, dal, mustard, green, chilli, dry, red, 2, cashew, or, peanuts, 1-1/2, tablespoon, oil, asafoetida]	[quantity, unit, ingredient, ingredient, quantity, ingredient, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient]

From the output above, we see that the rows for both columns have been split into their respective tokens, known as ‘input_tokens’ and ‘pos_tokens’. The tokens in ‘pos_tokens’ correspond to the tokens in ‘input_tokens’, eg: pos_tokens = ‘quantity’ -> input_tokens = 6, meaning that there are 6 tokens containing “quantity’.

2.2.2. Provide the length for input_tokens and pos_tokens and validate their length

1) An ‘input_length’ column is created in the DataFrame, which gives the length/number of the tokens that are present in the ‘input_tokens’ column. The length is calculated using the *len* in-built keyword and is applied using the *.apply()* function.
2) Similarly, a ‘pos_length’ column is created in the DataFrame, using the ‘pos_tokens’ column.
3) The columns are checked for their mismatched lengths with each other in a row-by-row fashion. This returns ‘True’ if the lengths are different and ‘False’ if they are the same.

```python
# create input_length and pos_length columns for the input_tokens and pos-tokens
df['input_length'] = df['input_tokens'].apply(len)
df['pos_length'] = df['pos_tokens'].apply(len)
```

```python
# check for the equality of input_length and pos_length in the dataframe
df[df['input_length'] != df['pos_length']]
```

	input	pos	input_tokens	pos_tokens	input_length	pos_length
17	2 cups curd 1 cup gourd cucumber green cor coriander 1/2 teaspoon cumin powder salt	quantity unit ingredient quantity unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient	[2, cups, curd, 1, cup, gourd, cucumber, green, cor, coriander, 1/2, teaspoon, cumin, powder, salt]	[quantity, unit, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient]	15	14
27	1 Baguette sliced 1 1/2 tablespoon Butter 1/2 Garlic minced cup Spinach Leaves Palak Red Bell pepper Capsicum Tomato finely chopped Onion Black powder Italian seasoning teaspoon Fresh cream Cheddar cheese grated Salt Roasted tomato pasta sauce	quantity ingredient ingredient quantity unit ingredient quantity ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[1, Baguette, sliced, 1, 1/2, tablespoon, Butter, 1/2, Garlic, minced, cup, Spinach, Leaves, Palak, Red, Bell, pepper, Capsicum, Tomato, finely, chopped, Onion, Black, powder, Italian, seasoning, teaspoon, Fresh, cream, Cheddar, cheese, grated, Salt, Roasted, tomato, pasta, sauce]	[quantity, ingredient, ingredient, quantity, unit, ingredient, quantity, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	37	36
79	1/2 cup Poha Flattened rice 2 tablespoons Rice flour 2 1/2 liter Milk 1 Nolen Gur or brown sugar Cardamom Elaichi Pods/Seeds 8-10 Mixed nuts almonds/cashews tablespoon Raisins pinch Saffron strands and a little more for garnish Salt	quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient quantity unit ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient unit ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[1/2, cup, Poha, Flattened, rice, 2, tablespoons, Rice, flour, 2, 1/2, liter, Milk, 1, Nolen, Gur, or, brown, sugar, Cardamom, Elaichi, Pods/Seeds, 8-10, Mixed, nuts, almonds/cashews, tablespoon, Raisins, pinch, Saffron, strands, and, a, little, more, for, garnish, Salt]	[quantity, unit, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, unit, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, unit, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	38	37

The output above shows the mismatched columns in the dataset. The mismatch is further shown between the 'input_length' and 'pos_length' columns where they have a difference of 1 label between them. Common causes for this difference could be short-forms being assigned 2 different tokens, fractions (eg: ½) being included in the recipe and composite ingredient names like 'almond/cashew'.

2.2.3. Define a unique_labels function and validate the labels in pos_tokens and Provide the insights seen in the recipe data after validation

1) A function named 'unique_labels' is created to check for all the unique POS labels that are present in the dataset.
2) An empty set, within the function, is created to store the unique labels while duplicates are automatically removed.
3) A for loop is used to iterate through the POS tokens where 'tag' is a list of labels. Hence, a nested for loop is used to iterate through this list.
4) The POS tag is added to the set using *.add()*. If it is already present, a duplicate will not be added to it. Finally, the set of labels is returned.

```
# Define a unique_labels function to checks for all the unique pos labels in the recipe & print it
def unique_labels(df):
    labels = set()
    for tags in df['pos_tokens']:
        for tag in tags:
            labels.add(tag)
    return labels

unique_labels(df)
```

```
{'ingredient', 'quantity', 'unit'}
```

Based on the output above, we see that the POS labels have been verified and identified as 3 unique labels, namely 'ingredient', 'quantity' and 'unit'. This was done as it is important for the model configuration process to know all the labels that are present in the dataset before training.

Insights from the previous step(s)

After splitting the input and pos columns into token lists and validating their lengths, we see that most rows do not have any mismatch, or have an alignment between input tokens and POS tags. Initially, a few rows had mismatches due to formatting issues in the raw data. These rows should be cleared from the dataset so that they do not impact future model configurations negatively.

2.2.4. Drop the rows that have invalid data provided in previous cell

1) Each row amongst the ‘input_length’ and ‘pos_length’ columns is compared against one another to find the mismatches.
   It returns True when the value in ‘input_length’ and the value in ‘pos_length’ match. Else, it returns False.
2) The *.reset_index()* function is used in resetting the row index after filtering out the old index. This is done to ensure that the dataset has been cleaned with a continuous index starting from 0.

```
# drop the irrelevant recipe data
df = df[df['input_length'] == df['pos_length']].reset_index(drop=True)
```

2.2.5. Update the input_length & pos_length in dataframe

This step is done to recompute and update the token and label counts present in the data after filtering in the previous section.

1) A new column, named ‘input_length’, is created by updating the ‘input_tokens’ columns through the *.apply()* function which applies the *len* keyword to each list and counts how many tokens are present.
2) Similarly, another new column named ‘pos_length’ is created which has the count of tokens present using the same function *.apply()*

```
# update the input and pos length in input_length and pos_length
df['input_length'] = df['input_tokens'].apply(len)
df['pos_length'] = df['pos_tokens'].apply(len)
```

2.2.6. Validate the input_length and pos_length by checking unequal rows

This is being done to check that every input has a corresponding label attached to it, which is mandatory for sequence-labelling tasks done by the model in the future. Hence, it can be considered as a validation check that the data has been filtered correctly.

1) The code checks each row using the Boolean comparison. It returns True if the number of input tokens are not equal to the length of POS labels. Else, it returns False (which means that each input token corresponds to its POS label).

```
# validate the input length and pos length as input_length and pos_length
df[df['input_length'] != df['pos_length']]
```

input	pos	input_tokens	pos_tokens	input_length	pos_length

From the output above, we see that there are no output values (except for the headers in the table). This means that there are no unequal rows present in the DataFrame after the removal process in the previous steps.

3. Train Validation Split (70 train - 30 val)

3.1. Perform train and validation split ratio

3.1.1. Split the dataset into train_df and val_df into 70:30 ratio

The splitting of the dataset is done so that the model can be trained on one portion while the other can be used to test the accuracy of the model.

1) The *train_test_split()* function is from the sci-kit learn package and it randomly splits the dataset into 2 separate subsets named 'val_df' and 'train_df'.
2) The *'test_size=0.3'* attribute is used to specify the size of the validation set. This means that 30% of the data will be considered as validation while the remaining 70% of the data is used for training.
3) The *'random_state=42'* attribute is used to set the random sampling seed for the dataset. This helps ensure that every split is reproducible.

```
# split the dataset into training and validation sets
train_df, val_df = train_test_split(df, test_size=0.3, random_state=42)
```

3.1.2. Print the first five rows of train_df and val_df

The *.head()* function is used on both 'train_df' and 'val_df'. By default, it shows the first 5 rows that are present in the DataFrame.

```
# print the first five rows of train_df
train_df.head()
```

	input	pos	input_tokens	pos_tokens	input_length	pos_length
175	250 grams Okra Oil 1 Onion finely chopped Tomato Grated teaspoon Ginger 2 Garlic Finely 1/2 Cumin seeds 1/4 Teaspoon asafoetida cup cottage cheese pinched coriander powder mango red chilli turmeric	quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient unit ingredient quantity ingredient ingredient quantity ingredient ingredient quantity unit ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[250, grams, Okra, Oil, 1, Onion, finely, chopped, Tomato, Grated, teaspoon, Ginger, 2, Garlic, Finely, 1/2, Cumin, seeds, 1/4, Teaspoon, asafoetida, cup, cottage, cheese, pinched, coriander, powder, mango, red, chilli, turmeric]	[quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, quantity, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	31	31
55	200 grams Paneer Homemade Cottage Cheese 2 Potato Aloo Bay leaf tej patta Dry Red Chilli 1 tablespoon Panch Phoran Masala roasted and powdered Tomato big sized teaspoon Turmeric powder Haldi Cumin seeds Jeera Ginger grated Salt 1/2 Sugar Sunflower Oil	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient	[200, grams, Paneer, Homemade, Cottage, Cheese, 2, Potato, Aloo, Bay, leaf, tej, patta, Dry, Red, Chilli, 1, tablespoon, Panch, Phoran, Masala, roasted, and, powdered, Tomato, big, sized, teaspoon, Turmeric, powder, Haldi, Cumin, seeds, Jeera, Ginger, grated, Salt, 1/2, Sugar, Sunflower, Oil]	[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient]	41	41
109	500 grams Cabbage Patta Gobi Muttaikose 1 teaspoon Mustard seeds 1-1/2 White Urad Dal Split sprig Curry leaves Green Chilli 1/4 cup Fresh coconut Salt	quantity unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient	[500, grams, Cabbage, Patta, Gobi, Muttaikose, 1, teaspoon, Mustard, seeds, 1-1/2, White, Urad, Dal, Split, sprig, Curry, leaves, Green, Chilli, 1/4, cup, Fresh, coconut, Salt]	[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient]	25	25

```
# print the first five rows of the val_df
val_df.head()
```

	input	pos	input_tokens	pos_tokens	input_length	pos_length
33	1 cup Ada 2 liter Milk 3/4 Sugar tablespoon Ghee 1/2 teaspoon Cardamom Powder Elaichi	quantity unit ingredient quantity unit ingredient quantity ingredient unit ingredient quantity unit ingredient ingredient ingredient	[1, cup, Ada, 2, liter, Milk, 3/4, Sugar, tablespoon, Ghee, 1/2, teaspoon, Cardamom, Powder, Elaichi]	[quantity, unit, ingredient, quantity, unit, ingredient, quantity, ingredient, unit, ingredient, quantity, unit, ingredient, ingredient, ingredient]	15	15
108	1 Carrot Gajjar chopped 7 Potatoes Aloo 2 cups Cauliflower gobi cut to small florets Onion tablespoon Ginger Garlic Paste Salt teaspoons Sunflower Oil 1/2 cup Fresh coconut grated teaspoon Whole Black Peppercorns Green Chillies Fennel seeds Saunf Poppy 6 Cashew nuts inch Cinnamon Stick Dalchini Star anise 3 Cloves Laung Cardamom Elaichi Pods/Seeds Cumin Jeera	quantity ingredient ingredient ingredient quantity ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient unit ingredient ingredient quantity unit ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[1, Carrot, Gajjar, chopped, 7, Potatoes, Aloo, 2, cups, Cauliflower, gobi, cut, to, small, florets, Onion, tablespoon, Ginger, Garlic, Paste, Salt, teaspoons, Sunflower, Oil, 1/2, cup, Fresh, coconut, grated, teaspoon, Whole, Black, Peppercorns, Green, Chillies, Fennel, seeds, Saunf, Poppy, 6, Cashew, nuts, inch, Cinnamon, Stick, Dalchini, Star, anise, 3, Cloves, Laung, Cardamom, Elaichi, Pods/Seeds, Cumin, Jeera]	[quantity, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	56	56
240	1 tablespoon Sunflower Oil 3 Potato Aloo Ginger paste Green Chilli chopped 1-1/12 tablespoons Sesame seeds Til teaspoon Red powder Cumin Jeera Coriander Powder Dhania 1/2 Garam masala 2 Sweet Chutney Date Tamarind Leaves few	quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient	[1, tablespoon, Sunflower, Oil, 3, Potato, Aloo, Ginger, paste, Green, Chilli, chopped, 1-1/12, tablespoons, Sesame, seeds, Til, teaspoon, Red, powder, Cumin, Jeera, Coriander, Powder, Dhania, 1/2, Garam, masala, 2, Sweet, Chutney, Date, Tamarind, Leaves, few]	[quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient]	35	35

Based on the outputs for both train_df and val_df, we see that the POS label and Input length for each row are the same. This confirms that the token-label alignment is correct and these rows can be used for model training and testing.

3.1.3. Extract the dataset into train_df and val_df into X_train, X_val, y_train and y_val and display their length

This step is done to convert the DataFrame columns into lists that are ready for training and evaluation by the model. The validation checks done here are to ensure that there is no difference in numeric values between the inputs and labels as if there were some difference to exist, the model would fail and produce incorrect results.

1) The 'input_tokens' column is selected from both the train_df and val_df, and it converts each row into a list of words using the *.tolist()* function. They are then stored in X_train and X_val variables, respectively.
2) Similarly, the 'pos_tokens' column is selected from both the train_df and val_df, converting them to a list using the *.tolist()* function. They are then stored in y_train and y_val variables respectively.
3) The shape of these samples are validated using the *len()* function as it counts the number of rows that are present in each set of DataFrame (X_train, X_val, y_train, y_val)

```
# extract the training and validation sets by taking input_tokens and pos_tokens
X_train = train_df['input_tokens'].tolist()
X_val = val_df['input_tokens'].tolist()

y_train = train_df['pos_tokens'].tolist()
y_val = val_df['pos_tokens'].tolist()

# validate the shape of training and validation samples
print(len(X_train), len(y_train))
print(len(X_val), len(y_val))
```

```
196 196
84 84
```

Based on the output above, we see that the training and validation sets have equal numbers of features.

The training set has 196 input sequences and 196 label sequences. Similarly, the validation set has 84 input sequences and 84 label sequences. This helps in confirming that each input sample has exactly 1 label to it and no data has been lost while transforming and splitting.

3.1.4. Display the number of unique labels present in y_train

This step is done to identify how many distinct output classes the model will need to predict and configure the final output layer size as well as

label mappings accordingly. It also checks if the training data has all the expected POS tags present.

1) The code uses nested for loops which iterates through each sentence (also known as a list of POS labels) in y_train and then through each POS tag within it. This accesses every individual label present in the DataFrame.
2) The POS labels are then flattened into a single collection and stored as a set using the *set()* function. This removes duplicates and then counts the unique POS labels that are used in training.

```
# Display the number of unique labels present in y_train
unique_train_labels = set(tag for sent in y_train for tag in sent)
len(unique_train_labels)
```

```
3
```

Based on the output above, we see that there are 3 unique labels present in y_train, which are *Ingredient, Quantity* and *Unit*. This has already been checked in section 2.2.3.

4. Exploratory Recipe Data Analysis on Training Dataset

4.1. Flatten the lists for input_tokens & pos_tokens

This step is required to flatten label sequences to easily analyse, count and/or visualise, as in the future model training, it may expect 1-dimensional lists instead of nested ones.

1) A function, named *flatten_list()*, has been defined to take a nested list as its argument. It also uses a list comprehension to loop though each inner list and its respective element. This results in a single dimensional list with all the elements from the nested structure.
   For example, a nested list containing [[‘quantity’, ‘unit’], ‘ingredient’] will be changed to a single list containing [‘quantity’, ‘unit’, ‘ingredient’].
2) The dataset is initialised and is given an identifier, ‘Training’, for it to be processed. This helps when distinguishing in logs and evaluation when performed on training or validation sets.

```
# flatten the list for nested_list (input_tokens, pos_tokens)
def flatten_list(nested_list):
    return [item for sublist in nested_list for item in sublist]
```

```
# initialise the dataset_name
dataset_name = 'Training'
```

4.2. Extract and validate the tokens after using the flattening technique

This step is done to flatten the tokens and labels across the entire dataset for easy validation check on token-label alignment. This is an important step before building and training the sequence-labelling model.

1) The code defines the function *extract_and_validate_tokens()* which takes 2 parameters - df and dataset_name. It is used to convert the 'input_tokens' column into a list and then flatten into a single dimensional list using the previously defined *flatten_list()* function.
2) The first 10 'input_tokens' list and flattened POS tokens are displayed and the function returns both flattened input tokens and POS tokens that are required for further steps.
3) The *extract_and_validate_tokens()* function is then called on the train_df and dataset. It is then split into 2 variables - 'train_input_tokens' and 'train_pos_tokens'.

```
# define a extract_and_validate_tokens with parameters (df, dataset_name)
# call the flatten_list and apply it on input_tokens and pos_tokens
# validate their length and display first 10 records having input and pos tokens
def extract_and_validate_tokens(df, dataset_name):
    input_tokens_flat = flatten_list(df['input_tokens'].tolist())
    pos_tokens_flat = flatten_list(df['pos_tokens'].tolist())

    print(f"{dataset_name} input tokens length:", len(input_tokens_flat))
    print(f"{dataset_name} pos tokens length:", len(pos_tokens_flat))

    print("First 10 input tokens:", input_tokens_flat[:10])
    print("First 10 pos tokens:", pos_tokens_flat[:10])

    return input_tokens_flat, pos_tokens_flat
```

```
# extract the tokens and its pos tags
train_input_tokens, train_pos_tokens = extract_and_validate_tokens(train_df, dataset_name)
```

```
Training input tokens length: 7114
Training pos tokens length: 7114
First 10 input tokens: ['250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated']
First 10 pos tokens: ['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']
```

Based on the output above, we see that there are a total of 7114 input and POS labels, respectively, confirming that each input token has its corresponding POS label. The first few words from the flattened training data are shown, such as '250', 'grass', 'okra', 'Oil', etc. Similarly, the corresponding POS labels are shown for the input tokens, such as 'quantity', 'unit', 'ingredient', 'ingredient'.

4.3. Categorise tokens into labels (unit, ingredient, quantity)

This section is used to categorize tokens into meaningful semantic categories and enables structured feature extraction from unstructured data.

1) A user-defined *categorize_tokens()* function is created with 2 input parameters being the list of tokens and POS tags.
2) 3 empty lists named ingredients, units and quantities are initialised to store their respective POS tags separately.
3) A for loop is used to iterate through the *.zip()* function that has an input token and its corresponding POS label. It checks the POS label and if it is 'ingredients', it is added to the ingredients list. Similarly, if it is 'units', it is added to the units list with the help of the *.append()* function.
4) The 3 types of tokens are converted to lowercase, using *.lower()*, to maintain consistency.
5) The function is called and the token-label pairs are extracted from the training dataset. It stores the returned values into 3 different variables - 'ingredient_list', 'unit_list' and 'quantity-list'. Each variable will contain the tokens that belong to its respective category.

```
def categorize_tokens(tokens, pos_tags):
    ingredients = []
    units = []
    quantities = []

    for token, tag in zip(tokens, pos_tags):
        if tag == 'ingredient':
            ingredients.append(token.lower())
        elif tag == 'unit':
            units.append(token.lower())
        elif tag == 'quantity':
            quantities.append(token.lower())

    return ingredients, units, quantities

#  call the function to categorise the labels into respective list
ingredient_list, unit_list, quantity_list = categorize_tokens(
    train_input_tokens, train_pos_tokens
)
```

4.4. Top 10 Most Frequent Items

A reusable function is created to allow for multiple analyses to be done on different categories. Essentially, it is used to help identify the top 10 frequently occurring items in the respective categories that were created in the previous section.

1) A user-defined *get_top_frequent_items()* function is created which takes 3 parameters - item_list, label and dataset_name.

2) A *Counter()* object is initialised within the function and it is used to count how many times each item appears in item_list. It then extracts the top 10 most frequent items using the *.most_common(10)* function.
3) The result is then displayed using a for loop to print the POS tag and its count side-by-side.
4) The function is called and the 'ingredient' and 'units' labels are passed into it to display the top 10 most frequent values for both categories.

```
def get_top_frequent_items(item_list, label, dataset_name):
    counter = Counter(item_list)
    top_items = counter.most_common(10)

    print(f"Top 10 {label}s in {dataset_name} dataset:")
    for item, count in top_items:
        print(item, ":", count)

    return top_items
```

```
topIngredients = get_top_frequent_items(ingredient_list, 'ingredient', dataset_name)
```

```
Top 10 ingredients in Training dataset:
powder : 192
salt : 123
leaves : 120
oil : 111
seeds : 111
red : 110
green : 104
chilli : 96
coriander : 93
chopped : 84
```

Output 1

```
topUnits = get_top_frequent_items(unit_list, 'unit', dataset_name)
```

```
Top 10 units in Training dataset:
teaspoon : 165
cup : 136
tablespoon : 100
grams : 63
tablespoons : 62
inch : 52
cups : 50
sprig : 42
cloves : 39
teaspoons : 39
```

Output 2

From output 1, we see the top 10 frequently occurring *INGREDIENTS* in the training dataset. This indicates the most commonly/frequently used ingredients

that are used in cooking, across all the recipes. For example: ‘powder’ occurs 192 times, meaning that it is a very common ingredient across most recipes.
Similarly from output 2, we see the top 10 frequently occurring *UNITS* (measuring units) in the training dataset. This indicates the most common or typically used units in recipes. For example: ‘teaspoon’ occurs 165 times, meaning that it is a very common measuring unit across most recipes.

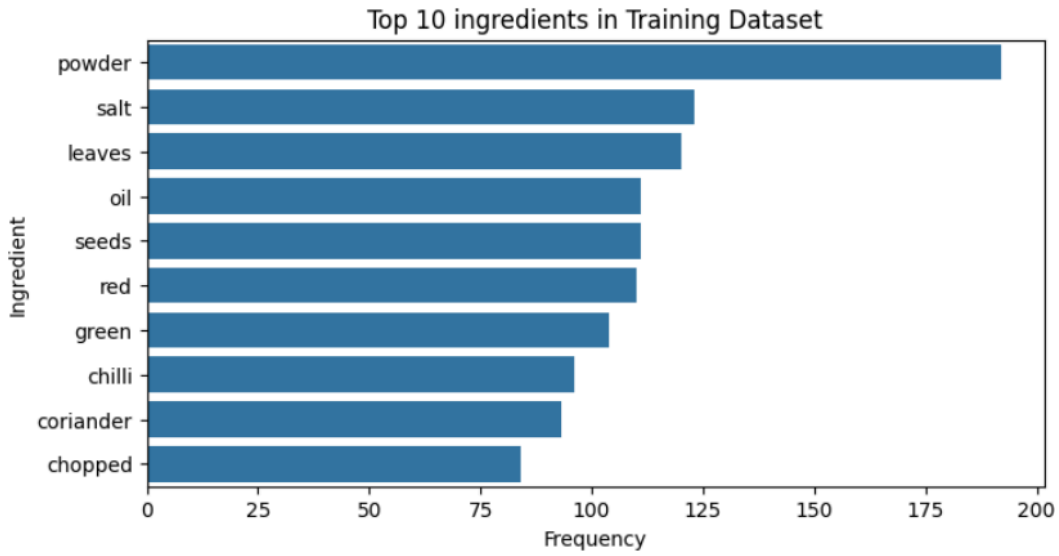
4.5. Plot Top 10 Most Frequent Items

1) A user-defined function, named *plot_top_items()*, is created and takes in 3 arguments - ‘top_items’, ‘labels’ and ‘dataset_name’.
2) The function extracts only the item names and its frequency counts that are present in the specified dataset.
3) These values are then plotted in a horizontal bar chart, using the *sns.barplot()* function. This is to help visualise and interpret the frequency counts better, rather than depending on raw numerical output. The frequency counts are on the x-axis while the item labels are on the y-axis.
4) The *plt.show()* function is used to display the bar chart after all the configurations are done when the function is called.

```
def plot_top_items(top_items, label, dataset_name):
    items = [item for item, _ in top_items]
    counts = [count for _, count in top_items]

    plt.figure(figsize=(8, 4))
    sns.barplot(x=counts, y=items)
    plt.title(f"Top 10 {label}s in {dataset_name} Dataset")
    plt.xlabel("Frequency")
    plt.ylabel(label.capitalize())
    plt.show()
```

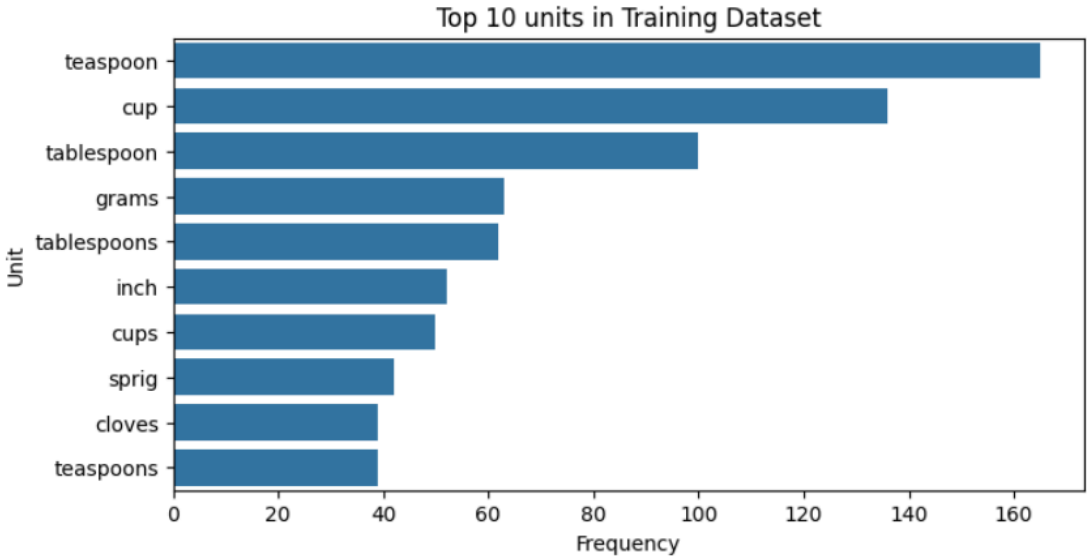
4.6. Perform EDA Analysis

```
# plot the top frequent ingredients in training data
plot_top_items(topIngredients, 'ingredient', dataset_name)
```



Based on the output above, we see that 'powder' is the most frequently used label/word in the recipe dataset.

```
# plot the top frequent units in training data
plot_top_items(topUnits, 'unit', dataset_name)
```



Similarly based on the output, we see that 'teaspoon' is the most frequently used label/unit of measurement in the recipe dataset.

5. Exploratory Recipe Data Analysis on Validation Dataset (Optional)

5.1. Execute EDA on Validation Dataset with Insights

This section validates that the validation dataset is clean and correctly labelled. It also ensures that the distributions are consistent with the training dataset. This will improve the model evaluation results and overall, the model will be unbiased and reliable.

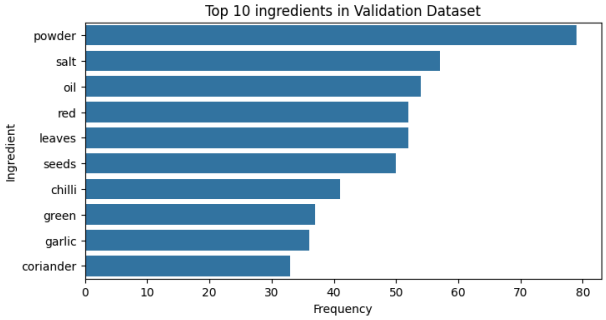
1) The name of the dataset is changed to 'Validation' for easy understanding.
2) The tokens are extracted from the validation dataset using the user-defined *extract_and_validate_tokens()* function. It verifies that both token lists have the same length and returns the flattened list of input tokens and their POS tags.
3) The validation tokens are then categorised .*8

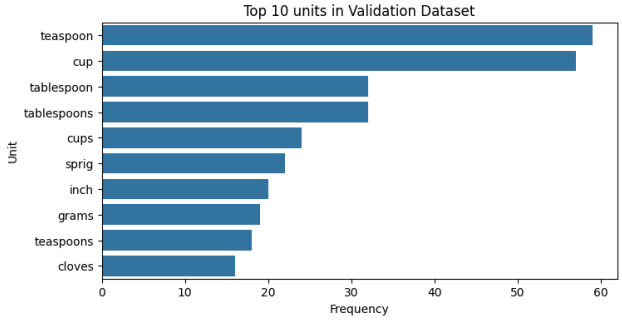
```
# initialise the dataset_name
dataset_name = 'Validation'
val_input_tokens, val_pos_tokens = extract_and_validate_tokens(val_df, dataset_name)

val_ingredient_list, val_unit_list, val_quantity_list = categorize_tokens(
    val_input_tokens, val_pos_tokens
)

val_top_ingredients = get_top_frequent_items(
    val_ingredient_list, 'ingredient', dataset_name
)

val_top_units = get_top_frequent_items(
    val_unit_list, 'unit', dataset_name
)
```

```
Validation input tokens length: 2876
Validation pos tokens length: 2876
First 10 input tokens: ['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']
First 10 pos tokens: ['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']
Top 10 ingredients in Validation dataset:
powder : 79
salt : 57
oil : 54
red : 52
leaves : 52
seeds : 50
chilli : 41
green : 37
garlic : 36
coriander : 33
Top 10 units in Validation dataset:
teaspoon : 59
cup : 57
tablespoon : 32
tablespoons : 32
cups : 24
sprig : 22
inch : 20
grams : 19
teaspoons : 18
cloves : 16
```

From the output above, we see that both the input and POS tokens are both 2876, which means that there is a perfect alignment between them. The sample tokens show the natural structure of the recipes, for example: [‘1’, ‘cup’, ‘Ada’] => [‘quantity’, ‘unit’, ‘ingredient’], which also shows that the POS labels are mapped correctly to the input tokens.
The top ingredients like powder, salt, oil, etc are frequently occurring, suggesting that it is a reusable term in different recipes. Similarly, common cooking measurements like teaspoon, cup, tablespoon, etc dominate the recipe dataset. However, there are minor variations (plural forms) in some words like tablespoon vs tablespoons.

To make it easier to visualise the top 10 items of ingredients and units categories, I have used a bar chart to represent them, as shown below.





6. Feature Extraction For CRF Model

6.1. Define a feature function to take each token from recipe

6.1.1. Define keywords for unit and quantity and create a quantity pattern to work on fractions, numbers and decimals

This section defines a foundation to accurately identify quantities and units in the recipe dataset. The defined sets are to capture common measurement terms and the regular expression set is used to handle numeric quantities that exist in different formats.

1) The ‘unit’ keywords are defined to create a set of measurement units that are commonly found within the recipe dataset. It includes all forms of the words - singular, plural, abbreviations.
   A set is created to allow fast and easy access to tokens.
2) Similarly, the ‘ingredient’ keywords store the word-based quantities such as ‘half’, ‘one’, ‘two’, etc. This helps in identifying quantified expressions that are in text forms.
3) A regex quantity pattern is created to match the numeric quantities, such as:

1, 2 => Whole Numbers
2/4, 1-2 => Fractions
1.5, 1.25 => Decimals
This helps to ensure flexible recognition of quantities in different formats.

4) The SpaCy model is loaded. It is an English NLP model that enables tokenisation, sentence parsing and linguistic analysis. The model is used to prepare the pipeline for processing complete recipe sentences.

```
# define unit and quantity keywords along with quantity pattern
unit_keywords = {
    'cup', 'cups', 'tablespoon', 'tablespoons', 'tbsp', 'teaspoon', 'teaspoons',
    'tsp', 'grams', 'gram', 'kg', 'ml', 'litre', 'liter', 'inch', 'inches',
    'cloves', 'sprig'
}

quantity_keywords = {
    'half', 'quarter', 'full', 'one', 'two', 'three'
}

quantity_pattern = re.compile(r'^\d+([/-]\d+)?(\.\d+)?$')
```

```
# load spaCy model
nlp = spacy.load("en_core_web_sm")
```

6.1.2. Define Feature Functions for CRF

The function defined here is used to convert each token into a set of features that are required for CRF training. It helps in identifying the ingredients, units and quantities accurately to combine the model features with its rules.

1) The *word2features()* function is defined to extract token-level features for a sentence. The parameters that are taken are *'sent'*, which is a list of tokens and *'i'*, which is the index/position of the current token. This formatting is required for CRF-sequence labelling models.
2) The list of tokens is converted into a full sentence string and it is processed with spaCy to obtain linguistic information. It then helps to extract the current token and its raw text.
3) A dictionary is created to store the core features that describe the token. These features help the CRF model to learn the token identity and grammar.
   "bias" =
   "tokens", "lemma" =
   "pos_tag", "tag", "dep" =
   "shape" =
   "Is_stop", "is_punct" =

4) The features then go through enhanced detection of recipe-specific entities to help improve recognition of quantities and measurement units.
   “is_quantity” = matches numeric patterns or keyword quantities
   “is_unit” = checks against known measurement units
   “is_numeric” = detects pure numbers
   “is_fraction” = identifies fractional quantities
   “is_decimal” = identifies decimal quantities
5) The if-else condition is used to add features from the previous token in its lower case form and if it represents a quantity or digit. If the token is the first in the sentence, it will be marked as Beginning of Sentence (BOS). The contextual words help the CRF model to capture sequence dependencies that are present in it.
6) Similarly, the other if-else condition adds features from the next token - its lowercase form, whether it's a unit or ingredient. If the token is the last in the sentence, it will be marked as End of Sentence (EOS). This helps identify patterns like quantity → unit → ingredient.
7) A structured feature dictionary is returned for one token. This will then be used by the CRF model during training and prediction.

```
def word2features(sent, i):
    doc = nlp(" ".join(sent))
    token = doc[i]
    word = token.text
    # Process the entire sentence with spaCy
    # --- Core Features ---
    features = {
        'bias': 1.0,
        'token': word.lower(),
        'lemma': token.lemma_.lower(),
        'pos_tag': token.pos_,
        'tag': token.tag_,
        'dep': token.dep_,
        'shape': token.shape_,
        'is_stop': token.is_stop,
        'is_digit': token.is_digit,
        'has_digit': any(char.isdigit() for char in word),
        'has_alpha': any(char.isalpha() for char in word),
        'hyphenated': '-' in word,
        'slash_present': '/' in word,
        'is_title': word.istitle(),
        'is_upper': word.isupper(),
        'is_punct': token.is_punct
    }
    # --- Improved Quantity & Unit Detection ---
```

```
    features.update({
        'is_quantity': (
            word.lower() in quantity_keywords or
            bool(quantity_pattern.match(word))
        ),
        'is_unit': word.lower() in unit_keywords,
        'is_numeric': word.isdigit(),
        'is_fraction': '/' in word,
        'is_decimal': '.' in word
    })
    # --- Contextual Features ---
    if i > 0:
        prev_word = sent[i - 1]
        features.update({
            'prev_token': prev_word.lower(),
            'prev_is_quantity': bool(quantity_pattern.match(prev_word)),
            'prev_is_digit': prev_word.isdigit()
        })
    else:
        features['BOS'] = True
    if i < len(sent) - 1:
        next_word = sent[i + 1]
        features.update({
            'next_token': next_word.lower(),
            'next_is_unit': next_word.lower() in unit_keywords,
            'next_is_ingredient': (
                next_word.lower() not in unit_keywords and
                not quantity_pattern.match(next_word)
            )
        })
    else:
        features['EOS'] = True
    return features
```

6.2. Preparation of recipe-level features

6.2.1. Define function to work to on all the recipes and call word2features for each recipe

The user-defined function is used to convert an entire sentence into a sequence of token-level representations which is required by CRF models when building and training. It ensures that each token is correctly processed and aligned with its label.

1) A *sent2features()* function is defined to process one complete sentence at a time.
2) The for loop in the list comprehension iterates over each token that is present using the *range(len())* functions.
   range() generates a sequence of numbers from 0 to a specified value.
   len() returns the total number of elements that are present
3) It then calls the user-defined *word2features()* function in the previous section to extract the feature dictionary and collects all the token-level features into a list.
4) The function then returns the list sequence of feature dictionaries that are representing the sentence.

```
def sent2features(sent):
    return [word2features(sent, i) for i in range(len(sent))]
```

6.3. Convert X_train, X_val, y_train and y_val into train and validation feature sets and labels

6.3.1. Convert recipe into feature functions by using X_train and X_val

This step is used to transform both datasets into a structured format while preserving the sentence as the CRF model is trained and validated on feature representations instead of raw tokens.

1) The list comprehension used for 'X_train_features' iterates over each sentence in the training dataset and calls the *sent2features()* function to convert all the tokens into feature dictionaries.
2) This stores the resulting feature sequences in the defined variable.
3) The same process is followed through for the validation dataset and the result is stored in the 'X_val_features' variable.
4) The output for both is a list of feature sequences that are suitable for CRF model training and evaluation.

```
X_train_features = [sent2features(sent) for sent in X_train]
X_val_features = [sent2features(sent) for sent in X_val]
```

6.3.2. Convert labels of y_train and y_val into a list

This step is done to standardise the label format and ensures compatibility with the feature sequences during the training and validation process. The CRF model will expect labels to be provided as lists of tags per sentence, which is a better format for processing.

1) The list comprehension for 'y_train_labels' iterates over each label sequence/sentence in the output/target training dataset (y_train).
2) It converts each sequence into a list to ensure a consistent structure in the format.

3) The same process is repeated for the validation dataset 'y_val' and stores the result in the 'y_val_labels'.
4) This ensures that the label sequences align correctly with their corresponding feature sequences.

```
# Convert labels into list as y_train_labels and y_val_labels
y_train_labels = [list(seq) for seq in y_train]
y_val_labels = [list(seq) for seq in y_val]
```

6.3.3. Print the length of val and train features and labels

This section verifies that the number of feature sequences matches the number of label sequences. It is done to ensure consistency before using the data to train and validate the CRF model.

1) The number of sentences in the training feature and training label sets are computed using the *len()* function.
2) The values are displayed side-by-side for easy comparison using the *print()* function.
3) This helps in ensuring that each feature sequence has its corresponding label sequence.

```
# print the length of train features and labels
print(len(X_train_features), len(y_train_labels))
```

```
196 196
```

```
# print the length of validation features and labels
print(len(X_val_features), len(y_val_labels))
```

```
84 84
```

Based on the outputs above, we see that there are 196 training sentences with each having a matching label sequence. Similarly, there are 84 validation sentences having their own corresponding label sequence.

6.4. Applying weights to feature sets

6.4.1. Flatten the labels of y_train

This section is done to flatten the labels for analysing individual token level, which is useful for some tasks.

1) A nested for loop is used where the outer loop iterates over each label sequence in 'y_train_labels' and the inner loop iterates again over each individual label inside that same sequence.
2) It then extracts every label and stores the POS labels into a single list named 'y_train_flat'.

```
# Flatten labels in y_train
y_train_flat = [label for seq in y_train_labels for label in seq]
```

6.4.2. Count the labels present in training target dataset

This section helps in understanding the distribution of POS labels in the training data by calculating the total samples present. This is necessary for detecting class imbalance before model training.

1) The *Counter()* object is initialised and is used to count how many times each label appears in the flattened dataset.
2) The frequencies are then stored in 'label_counts' as key-value pairs format, which in this case as label-count pairs.
3) The *.values()* function gets the number of labels (values) present in the flattened dataset.
4) Finally, the *sum()* function is then used to add up all the occurrences of the labels and get the total number of tokens.

```
label_counts = Counter(y_train_flat)
total_samples = sum(label_counts.values())
```

6.4.3. Compute weight_dict by using inverse frequency method for label weights

1) The class weights are computed with the inverse frequency method using "label_counts" and "total_counts".
2) The weight is calculated using the formula 'total_samples/count'. This implies that rare classes get higher weights while the frequent classes get lower weights.
3) If the label is "ingredient", the weight is increased by 50% from the original (hence, it is multiplied by 1.5). This step gives extra importance to the "ingredient" label during dataset training.

```
weight_dict = {
    label: total_samples / count
    for label, count in label_counts.items()
}
```

```
# penalise ingredient label
if 'ingredient' in weight_dict:
    weight_dict['ingredient'] *= 1.5
```

6.4.4. Extract features along with class weights

1) A function named *extract_features_with_class_weights()* is created and it takes 3 parameters:
   X = List of feature dictionaries for each token
   y = Corresponding labels for each token
   weight_dict = Dictionary that maps the class labels to their weights

2) An empty list named ‘weighted_features’ is created to store the sentences after adding the class weights.
3) An outer for loop is used to iterate over features (sentences) and labels simultaneously. Then an inner for loop is used to iterate over the token's features and its corresponding label.
4) The features are copied so that the original ones are unchanged, using the *.copy()* function.
5) The new key is added with the weight for the current label and will default to 1.0 if the label is not in the ‘weight_dict’. Then, both the token and sentence's weighted features are appended to their final lists - ‘sent_weighted’ and ‘weighted_features’ respectively, using the *.append()* function.

```
def extract_features_with_class_weights(X, y, weight_dict):
    weighted_features = []

    for sent_features, sent_labels in zip(X, y):
        sent_weighted = []
        for features, label in zip(sent_features, sent_labels):
            features_copy = features.copy()
            features_copy['class_weight'] = weight_dict.get(label, 1.0)
            sent_weighted.append(features_copy)
        weighted_features.append(sent_weighted)

    return weighted_features
```

6.4.5. Execute extract_features_with_class_weights on training and validation datasets

This section is done to ensure that the model learns from both training and validation datasets, and evaluates fairly while giving more importance to the rare classes.

1) The *extract_features_with_class_weights()* function is called on the training features and labels by adding the class weights from 'weight_dict' to each token's feature dictionary.
2) Similarly and in the same process, the class weights are applied to the validation dataset so that it is weighted consistently with the training set.

```
# Apply manually computed class weights
X_train_weighted_features = extract_features_with_class_weights(
    X_train_features, y_train_labels, weight_dict
)

X_val_weighted_features = extract_features_with_class_weights(
    X_val_features, y_val_labels, weight_dict
)
```

7. Model Building and Training

7.1. Initialise the CRF model and train it

This section shows the initialisation process of the Conditional Random Fields (CRF) model.

1) The *sklearn_crfsuite.CRF()* object, from the 'sklearn_crfsuite' package, initialises the object for CRF model with the following parameters:
   'algorithm' = The type of algorithm used on the model training for optimisation, which in this case is L-BFGS.
   'c1' = L1 regularisation coefficient to help sparsity of weights.
   'c2' = L2 regularisation coefficient to prevent overfitting of the model.
   'max_iterations' = The maximum number of iterations that can be done for training the model.
   'all_possible_transitions' = Allows the model to consider all the possible label transitions which helps in improving the generalisation of the model to new and unseen data in the future.
2) The model is then trained using the *.fit()* function with the parameters being 'X_train_weighted_features' - which are token-level features that include class weights, and 'y_train_labels' - which are the corresponding token-level labels.

```
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.5,
    c2=1.0,
    max_iterations=100,
    all_possible_transitions=True
)
# train the CRF model with the weighted training data
crf.fit(X_train_weighted_features, y_train_labels)
```

```
CRF
CRF(algorithm='lbfgs', all_possible_transitions=True, c1=0.5, c2=1.0,
    max_iterations=100)
```

The output above shows the trained CRF model. During the training process, the model learns the probabilities and feature weights to get the maximum likelihood of having the correct label sequences.

7.2. Evaluation of Training Dataset using CRF model

1) The trained CRF model makes predictions on the training dataset features using the *.predict()* function. The resulting 'y_pred_train' list gives

an output of predicted label sequences, where it matches the shape of the 'y_train_labels'.

2) A flat classification report, using *flat_classification_report()*, is created by flattening the sequences so it treats every token as an independent classification sample. The report contains precision, recall, F1-score and the support values for each class present in the dataset.
3) The confusion matrix is created using *confusion_matrix()* and it flattens all the token sequences into a single list of labels using the list comprehensions.
   The first list comprehension, *'label for seq in y_train_labels for label in seq'*, is used to get all the true labels present. The second list comprehension, *'label for seq in y_pred_train for label in seq'* is used to get all the predicted labels - both true and false that are present.

```
# evaluate on the training dataset
y_pred_train = crf.predict(X_train_weighted_features)
```

Code 1

```
# specify the flat classification report by using training data for evaluation
print(flat_classification_report(y_train_labels, y_pred_train))
```

Code 2

```
# create a confusion matrix on training datset
train_conf_matrix = confusion_matrix(
    [label for seq in y_train_labels for label in seq],
    [label for seq in y_pred_train for label in seq]
)
train_conf_matrix
```

Code 3

```
              precision    recall  f1-score   support

  ingredient       1.00      1.00      1.00      5323
    quantity       0.99      0.98      0.99       980
        unit       0.98      0.99      0.98       811

    accuracy                           1.00      7114
   macro avg       0.99      0.99      0.99      7114
weighted avg       1.00      1.00      1.00      7114
```

Output for Code 2

```
array([[5323,    0,    0],
       [   0,  963,   17],
       [   0,    9,  802]])
```

Output for Code 3

From the output for code 2, we see that there are 4 metrics used:
'precision' = number of tokens that were correctly predicted from all tokens present.
'recall' = number of tokens that were correctly predicted from all the true tokens present.
'f1-score' = mean of precision and recall.
'support' = number of tokens in the class of the dataset.
From these values, we see that the model performs well on the training dataset and especially well on the "ingredient" label. The accuracy of the model's classification is a perfect 1.0, implying that it is able to classify 100% of the labels correctly. The macro and weighted averages summarise the performance across all classes.

From the output for code 3, we see that the rows are the actual class labels and the columns are the predicted class labels in the "ingredient", "quantity", "unit" format. In the first row, we see that 5323 "ingredient" tokens have been correctly predicted. In the second row, we see that 963 "quantity" tokens have been correctly predicted and 17 "quantity" units have been misclassified as "unit". In the third row, we see that 9 unit tokens have been wrongly predicted as "quantity" while 802 "unit" tokens have been predicted correctly by the model. This shows that the model is highly accurate on training data with a few misclassifications.

7.3. Save the CRF model

The trained model is saved as 'crf_model.pkl' using the *joblib.dump()* object. The .pkl extension for the file stands for 'Pickle' which is Python's way of serialising the object. It is stored in a byte stream and can be reloaded exactly as it already was.

```
# dump the model using joblib as crf_model.pkl
joblib.dump(crf, 'crf_model.pkl')
```

```
['crf_model.pkl']
```

8. Prediction and Model Evaluation

8.1. Predict and Evaluate the CRF model on validation set

This section is similar to that of the process on the training dataset.

1) The trained CRF model makes predictions on the training dataset features using the *.predict()* function. The resulting 'y_pred_val' list gives an output of predicted label sequences.
2) A flat classification report, using *flat_classification_report()*, is created by flattening the sequences so it treats every token as an independent

classification sample. The report contains precision, recall, F1-score and the support values for each class present in the dataset.

3) The confusion matrix is created using *confusion_matrix()* and it flattens all the token sequences into a single list of labels using the list comprehensions.
   The first list comprehension, *'label for seq in y_val_labels for label in seq'*, is used to get all the true labels present. The second list comprehension, *'label for seq in y_pred_val for label in seq'* is used to get all the predicted labels - both true and false that are present.

```
# predict the crf model on validation dataset
y_pred_val = crf.predict(X_val_weighted_features)
```

Code 1

```
# specify flat classification report
print(flat_classification_report(y_val_labels, y_pred_val))
```

Code 2

```
# create a confusion matrix on validation dataset
val_conf_matrix = confusion_matrix(
    [label for seq in y_val_labels for label in seq],
    [label for seq in y_pred_val for label in seq]
)
val_conf_matrix
```

Code 3

```
              precision    recall  f1-score   support

  ingredient       1.00      1.00      1.00      2107
    quantity       0.98      0.97      0.98       411
        unit       0.97      0.98      0.97       358

    accuracy                           0.99      2876
   macro avg       0.98      0.98      0.98      2876
weighted avg       0.99      0.99      0.99      2876
```

Output for Code 2

```
array([[2107,    0,    0],
       [   0,  400,   11],
       [   0,    8,  350]])
```

Output for Code 3

From the output for code 2, we see that there are the similar 4 metrics used: 'precision' = number of tokens that were correctly predicted from all tokens present.

‘recall’ = number of tokens that were correctly predicted from all the true tokens present.
‘f1-score’ = mean of precision and recall.
‘support’ = number of tokens in the class of the dataset.
From this, we can interpret that 'ingredient' has perfect precision, recall and f1-score meaning that they were all correctly predicted. Similarly, 'quantity', 'unit' have high precision/recall but with a few misclassifications between them.

From the output for code 3, we can interpret that the diagonal elements are the number of tokens correctly predicted per class and the off-diagonal elements are the misclassifications that were done. This allows the understanding of where the model has made mistakes in validation.

9. Error Analysis on Validation Data

9.1. Investigate misclassified samples in validation dataset

9.1.1. Flatten the labels of validation data and initialise error data

This section is done to make the true and predicted labels comparable at its token level for evaluation and error analysis.

1) The true validation labels are flattened and are stored as a single list of sequences/labels, one label per token.
2) The predicted validation labels are flattened in the same way as the previous step and this ensures the true, predicted labels have the same structure and order.
3) An empty list named “error_data” is initialised to store the error information such as misclassified tokens.

```
# flatten Labels and Initialise Error Data
y_val_flat = [label for seq in y_val_labels for label in seq]
y_pred_flat = [label for seq in y_pred_val for label in seq]

error_data = []
```

9.1.2. Iterate the validation data and collect error information

This section is done to analyse the misclassifications identified in the previous section with both metrics and context. It is to ensure systematic errors can be found and improve features, labels or class weights that are present.

1) The outer for loop is used to iterate over each validation sentence and the model's predicted labels together.
2) The inner for loop iterates through each token in the sentence with its true label (true), its predicted label (pred) and "i" is the index position of the token.
3) The if condition is used to check if the token is misclassified and if it is, the prediction is then analysed further.

4) The previous token is retrieved if it exists and uses None to signify the start of the sentence. Similarly, the next token is retrieved if it exists and uses None to signify the end of the sentence.
5) The 'error_data' list is appended, using the *.append()* function, to store more detailed information about each error such as: misclassified token, surrounding context, true vs predicted label, importance of the class (weight) and full sentence context.

```
# iterate and collect Error Information

            # get previous and next tokens with handling for boundary cases
for sent_tokens, true_labels, pred_labels in zip(X_val, y_val_labels, y_pred_val):
    for i, (token, true, pred) in enumerate(zip(sent_tokens, true_labels, pred_labels)):
        if true != pred:
            prev_token = sent_tokens[i - 1] if i > 0 else None
            next_token = sent_tokens[i + 1] if i < len(sent_tokens) - 1 else None

            error_data.append({
                'token': token,
                'prev_token': prev_token,
                'next_token': next_token,
                'true_label': true,
                'predicted_label': pred,
                'class_weight': weight_dict.get(true, 1.0),
                'context': " ".join(sent_tokens)
            })
```

9.1.3. Create dataframe from error_data and print overall accuracy

1) The 'error_data' list is converted to a DataFrame using *pd.DataFrame()* to easily inspect, sort and analyse any model error.
2) The true and predicted labels are compared against each other at its token level and the accuracy is calculated using the formula: sum of correct predictions/total number of tokens.

```
# Create DataFrame and Print Overall Accuracy
error_df = pd.DataFrame(error_data)

accuracy = sum(
    t == p for t, p in zip(y_val_flat, y_pred_flat)
) / len(y_val_flat)

print("Validation Accuracy:", accuracy)
```

```
Validation Accuracy: 0.9933936022253129
```

Based on this, we see that the model correctly predicted 99.34% of all tokens in the validation set. There is a small difference from 100% accuracy which indicates a few misclassifications. Hence, this shows that the model shows strong generalisation to the data.

9.1.4. Analyse errors by label type

1) An empty dictionary named "label_error_analysis" is initialised to store per-label performance metrics.
2) A for loop is used to iterate over each unique label in the validation data using the *set()* function.
3) Within the for loop, the total number of the label appearing in the validation set is counted using the *sum()* function. The correct predictions are then counted in the model with the same label.
4) The label-specific accuracy using the formula: (correct/total), is computed and stored in the class weight used for the label during training.

```
label_error_analysis = {}

for label in set(y_val_flat):
    total = sum(1 for t in y_val_flat if t == label)
    correct = sum(
        1 for t, p in zip(y_val_flat, y_pred_flat) if t == p == label
    )
    label_error_analysis[label] = {
        'accuracy': correct / total if total > 0 else 0,
        'class_weight': weight_dict.get(label, 1.0)
    }

label_error_analysis
```

```
{'ingredient': {'accuracy': 1.0, 'class_weight': 2.0046965996618447},
 'unit': {'accuracy': 0.9776536312849162, 'class_weight': 8.771886559802713},
 'quantity': {'accuracy': 0.9732360097323601,
  'class_weight': 7.259183673469388}}
```

Based on the output above, we see that “ingredient” token has the highest accuracy of 100%, implying that all the ingredient tokens were correctly predicted. It also has the lowest class weight which implies that it is the most frequent class that occurs within the dataset.
Similarly, the “unit” token has the next highest accuracy of 97.8% implying that it had misclassified a few tokens. It has the highest class weight, meaning that it was a rare class and hence, it was difficult for the model to predict.
The “quantity” token has the lowest accuracy of 97.3%, implying that there were more tokens that were misclassified compared to the other 2. It also has a high class weight, meaning that it was less frequent and difficult for the model to predict.

9.2. Provide Insights from Validation Dataset

- From the validation performance, we see that the model achieves a very high accuracy of 99.34%, suggesting that the dataset is able to generalise

well to new and unseen data, and has minimal overfitting when compared to training performance.

- We also see that:
    - Ingredient labels have been perfectly predicted on the validation set. This implies that the tokens are distinct and can be easily learned by the model.
    - Quantity and unit labels have a slightly lower accuracy which could imply that these classes are harder to distinguish when appearing in similar contexts.
- The higher class weights for “unit” and “quantity” labels reflect the rare occurrences in the dataset. However, the model is still able to perform well showing that this method was effective.
- We also see that the misclassifications occurring in the dataset are not widespread and affect the accuracy of the model greatly.
- Overall, the validation results show that the CRF model is well-trained for imbalance data and suitable for real-world problems.

10. Conclusion (Optional)

- The overall project has demonstrated how effective the Conditional Random Fields (CRF) model is for sequence labelling in identifying key entities from an unstructured dataset.
- The data cleaning and validation that was initially done has enabled the dataset to be reliable in sequence modelling due to the token-label alignment that was achieved.
- The linguistic and contextual features have enabled the CRF model to accurately capture meaningful patterns in the dataset. An example would be the "quantity -> unit -> ingredient" sequence that was identified in the recipe dataset.
- We also can conclude that the CRF model was trained well as it has achieved excellent results and performed well/generalised well to new and unseen data (validation dataset). The validation set results produced a 99.34% accuracy on it.
- Despite the model’s high accuracy percentage, we can see that most of the misclassifications occurred with the "quantity" and "unit" labels, which show that the model is not perfect and has room for improvement.
- In conclusion, the results prove that a feature-based CRF model is suitable for structured information extraction in domain-specific NLP tasks.